Chapter 8: Stochastic Vehicle Routing Problems

Vehicle Routing: Problems, Methods, and Applications
2nd Edition, 2014

M. Gendreau O. Jabali W. Rei

Slides prepared from the textbook chapter

Table of Contents

What Is a Stochastic VRP? I

Vehicle Routing Problems (VRPs) have been the subject of massive research effort since Dantzig and Ramser introduced them in 1959. The vast majority of that work assumes *complete and certain information*: all customer locations, demands, and travel times are known in advance.

The Real-World Gap

In practice, a company may not know exactly how much product each customer will want, whether a customer will actually be at home, or how long the drive will take through unpredictable traffic. Planning ignores this uncertainty at its peril.

A **Stochastic Vehicle Routing Problem (SVRP)** is a VRP in which one or more problem parameters are random variables whose values are not fully known when the route plan is created. The probability distributions of these random variables are assumed to be known (or estimable from historical data).

What Is a Stochastic VRP? II

Three Main Sources of Stochasticity

1. **Stochastic demands** — the quantity ξ_i (xi sub i, pronounced "ksee") that customer i actually requires is a random variable. Its value is revealed only when the vehicle *arrives* at customer i.
2. **Stochastic customers** — not every customer who placed an order will necessarily be present. Each customer i is present with probability p_i (p sub i), and absent with probability $1 - p_i$.
3. **Stochastic travel times** — the time t_{ij} (t sub i-j) to travel from location i to location j is random, making it uncertain whether a vehicle will arrive within a customer's time window.

These three categories are not mutually exclusive; a real problem may involve any combination of them.

What Is a Stochastic VRP? III

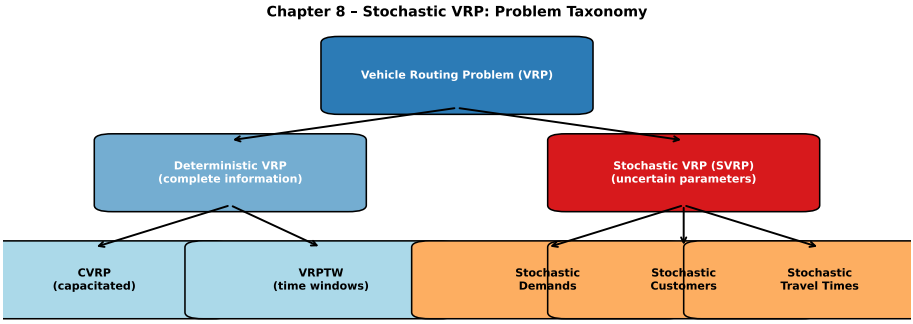


Figure: Taxonomy of Vehicle Routing Problems: deterministic vs. stochastic variants.

What Is a Stochastic VRP? IV

The key insight is that *uncertainty forces us to think in terms of probability distributions and expected values rather than single fixed numbers*. A good solution under stochastic conditions is one that performs well on average across all the possible realizations of the random parameters.

Two Modelling Paradigms I

When randomness is present, there are two fundamentally different ways to model the problem and construct a solution.

Paradigm 1: A Priori Optimization

Build a complete route plan *before* any information about the random parameters is revealed. The plan is a fixed schedule: vehicle 1 visits customers $\{3, 5, 7\}$, vehicle 2 visits $\{1, 2, 4, 6\}$, and so on. When reality unfolds and some demands are higher than expected, a simple *recourse rule* is applied (for example, return to the depot to reload), but the sequence of customers is otherwise unchanged.

The a priori approach suits situations where communication during route execution is difficult or expensive, or where the plan must be submitted to customers in advance.

Two Modelling Paradigms II

Paradigm 2: Reoptimization

Begin with an initial plan, but allow it to be modified *during route execution* as new information arrives. When the vehicle reaches customer i and the actual demand is revealed, a (fast) optimization is run to adjust the remaining route. This is also called an *online* or *dynamic* approach.
Reoptimization can produce much better solutions but requires real-time computation and communication.

Two Modelling Paradigms III

A third modelling choice concerns the *objective function*: should we minimize the *worst-case* cost, the *expected* cost, or require that certain constraints hold with high probability?

Probabilistic (Expected-Cost) Formulation

Minimize the *expected total route cost* over all possible realizations of the random parameters. This is the most common objective in the SVRP literature and leads to stochastic programming formulations.

Chance-Constrained Formulation

Require that certain constraints (e.g., "the vehicle must not run out of capacity") are satisfied with at least some probability $1 - \varepsilon$ (1 minus epsilon, where ε is a small threshold such as 0.05). This trades a small risk of infeasibility for a potentially lower planned cost.

The rest of this chapter examines these paradigms and formulations in detail, starting with a priori optimization.

The A Priori Paradigm I

In the a priori paradigm a complete route plan is constructed *before* the realization of any random variables is observed. Once the vehicle starts its tour and reality unfolds, a pre-specified *recourse policy* handles any deviations from the plan.

Formal Setup (Network)

We are given a complete undirected graph $G = (V, E)$, where:

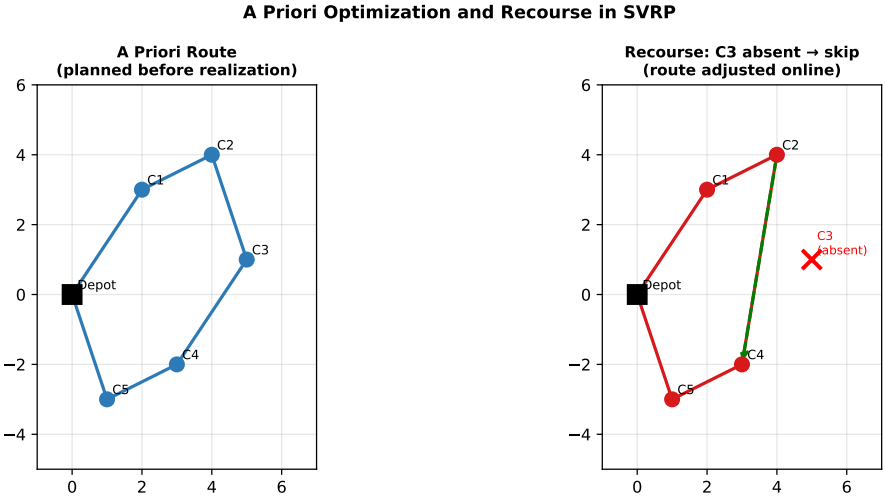
- $V = \{0, 1, 2, \ldots, n\}$ is the set of *vertices*. Vertex 0 is the depot; vertices $1, \ldots, n$ are customers.
- E is the set of *edges*: every pair of vertices is connected (the graph is complete).
- Each customer $i \in V \setminus \{0\}$ has a random demand ξ_i (xi sub i), drawn independently from a known distribution with mean μ_i (mu sub i).
- A single vehicle of capacity Q (maximum load it can carry) serves customers.

The A Priori Paradigm II

Definition: A Priori Solution

An *a priori solution* is a permutation $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n)$ (sigma, pronounced "sig-ma") listing all customers in the order they would be visited if every customer were present and had its expected demand. After the random variables are realized, the actual route is a *sub-sequence* of σ, with recourse actions inserted where needed.

The A Priori Paradigm III



The A Priori Paradigm IV

Figure: Left: the planned a priori route visiting all 5 customers in order. Right: recourse action (customer 3 absent — skip it and re-link the route).

The A Priori Paradigm V

Recourse Policy for Stochastic Demands

The most common recourse policy for stochastic demands is **re-supply**: if the vehicle arrives at customer i and the realized cumulative demand exceeds the vehicle capacity Q, the vehicle returns to the depot to reload, then continues the tour from customer i.
Formally, define the *failure event* at position k in the tour as:

$$\text{Fail}_k = \left\{ \sum_{j=1}^{k} \xi_{\sigma_j} > Q \right\}$$

That is, Fail_k occurs when the sum of realized demands for the first k customers in the tour exceeds the vehicle capacity Q. When this happens, the vehicle performs a *return trip* to the depot (cost: $2\, d_{0,\sigma_k}$, where d_{0,σ_k} is the distance from the depot to customer σ_k).

Here: ξ_{σ_j} (xi sub sigma-j) is the realized demand of the j-th customer visited; Q is the vehicle capacity; d_{0,σ_k} (d sub $0, \sigma_k$) is the travel distance from depot to customer σ_k.

The A Priori Paradigm VI

Key Insight

The a priori tour is evaluated under many random demand scenarios. Its quality is measured by the *expected total distance*, including the expected number and cost of recourse (restocking) trips. A tour that looks short in the deterministic case may be very costly under stochasticity if it passes through high-variance customers late in the route.

Network Flow Formulation for A Priori SVRP I

We now present the mathematical programming model for the a priori SVRP. The model determines which arcs (road segments) the vehicle uses in its planned tour while minimizing expected total travel cost including recourse.

Decision Variables

- $x_{ij} \in \{0, 1\}$ — "x sub *i*-*j*" equals 1 if the route goes directly from customer i to customer j, and 0 otherwise. These are the *arc selection* variables.
- $y_i \geq 0$ — "y sub i" represents the demand served at customer i (the decision of whether and how much to deliver).

Network Flow Formulation for A Priori SVRP II

Objective Function (8.6)

$$\min \sum_{(i,j)\in E} c_{ij}\, x_{ij} \;+\; \mathbb{E}\big[Q(\mathbf{x}, \boldsymbol{\xi})\big] \tag{8.6}$$

- c_{ij} (c sub *i*-*j*) — the *deterministic* travel cost from node i to node j (e.g. distance or time).
- x_{ij} — arc-selection binary variable (1 if arc used, 0 if not).
- $\mathbb{E}[\cdot]$ (E with double-bar, meaning "expected value") — the expectation operator taken over all possible demand realizations $\boldsymbol{\xi}$ (bold xi = vector of all customer demands $(\xi_1, \xi_2, \ldots, \xi_n)$).
- $Q(\mathbf{x}, \boldsymbol{\xi})$ — the *recourse cost function*: the additional cost incurred (due to restocking trips) when demand realization $\boldsymbol{\xi}$ occurs and the vehicle follows the arc pattern $\mathbf{x}$.

Reading this formula: the objective says "choose the arcs $\mathbf{x}$ to minimize the planned route cost plus the *expected* cost of all emergency restocking trips." Adding more slack (extra capacity margin) reduces expected recourse cost but may increase the base route cost — so the formula balances route efficiency against robustness.

Network Flow Formulation for A Priori SVRP III

Constraints

$$\sum_{j \in V} x_{ij} = 1 \quad \forall i \in V \setminus \{0\} \tag{8.7}$$

$$\sum_{i \in V} x_{ij} = 1 \quad \forall j \in V \setminus \{0\} \tag{8.8}$$

$$\sum_{j \in V} x_{0j} = \sum_{i \in V} x_{i0} = m \tag{8.9}$$

$$x_{ij} \in \{0, 1\} \tag{8.10}$$

Reading each constraint in plain words:

- **(8.7)** — Each customer i has exactly one arc *leaving* it (the vehicle departs customer i once).
- **(8.8)** — Each customer j has exactly one arc *entering* it (the vehicle arrives at customer j once).

Network Flow Formulation for A Priori SVRP IV

- **(8.9)** — The depot is left and re-entered exactly m (em) times, where m is the number of vehicles (routes).
- **(8.10)** — Arc variables are binary (0 or 1).

Sub-tour elimination constraints (not shown here) ensure the arcs form a valid set of complete routes from and to the depot, rather than disconnected cycles.

Network Flow Formulation for A Priori SVRP V

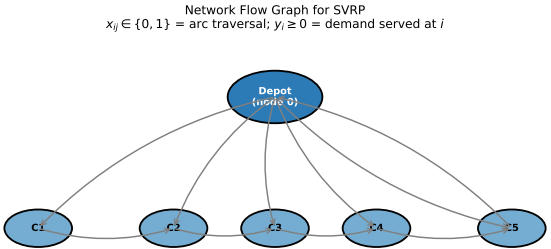


Figure: Network flow graph for the SVRP formulation. Arcs represent potential route segments; the depot is the central node.

The Set Partitioning Formulation I

An alternative, and computationally very powerful, way to state the a priori SVRP is the **set partitioning formulation**. Instead of working with individual arcs, we enumerate possible *complete routes* and select the best combination.

Intuition

Imagine listing every valid route a single vehicle could take (respecting capacity, visiting a subset of customers, starting and ending at the depot). Let Ω (Omega, the last letter of the Greek alphabet) be the set of all such routes. For each route $r \in \Omega$, let:

- c_r — the deterministic travel cost of route r.
- $Q_r(\boldsymbol{\xi})$ — the random recourse cost incurred on route r when demands are $\boldsymbol{\xi}$.
- $a_{ir} \in \{0, 1\}$ — (a sub i-r) equals 1 if route r includes customer i, and 0 otherwise.

The Set Partitioning Formulation II

Set Partitioning Model (8.10)–(8.12)

$$\min_{\lambda} \quad \sum_{r \in \Omega} \left(c_r + \mathbb{E}[Q_r(\boldsymbol{\xi})] \right) \lambda_r \tag{8.10}$$

$$\text{s.t.} \quad \sum_{r \in \Omega} a_{ir} \, \lambda_r = 1 \quad \forall i \in V \setminus \{0\} \tag{8.11}$$

$$\lambda_r \in \{0, 1\} \quad \forall r \in \Omega \tag{8.12}$$

Symbol explanations:

- λ_r (lambda sub r, pronounced "lam-da") — binary decision variable: equals 1 if route r is selected, 0 otherwise.
- $\sum_{r \in \Omega}$ (sum over all routes in Omega) — we add up the cost of every selected route.
- Constraint (8.11) says: every customer i must be covered by *exactly one* selected route. This is the "partitioning" condition.

The Set Partitioning Formulation III

Key Insight

The set partitioning formulation separates the problem into (1) evaluating individual routes and (2) choosing the best combination. This is ideal for branch-and-price algorithms, which generate routes on demand. The main challenge is that $|\Omega|$ (the number of feasible routes) can be astronomically large.

Computing the Expected Recourse Cost I

The expected recourse cost $\mathbb{E}[Q(\mathbf{x}, \boldsymbol{\xi})]$ is the heart of the SVRP objective. We now show how to compute it for a single route.

Recourse Cost for One Route

Consider a single route visiting customers $1, 2, \ldots, n$ in that order (for notational simplicity). Define $D_k = \sum_{i=1}^{k} \xi_i$ (D sub k, the cumulative demand after visiting the first k customers). The vehicle must restock whenever D_k first exceeds Q.
The expected recourse cost is:

$$\mathbb{E}[Q(\sigma, \boldsymbol{\xi})] \;=\; 2 \sum_{k=1}^{n} d(v_k, 0)\, \mathbb{E}\Big[\mathbf{1}\Big\{ D_k > Q \cdot \Big\lfloor D_{k-1}/Q \Big\rfloor + Q \Big\}\Big]$$

where $\mathbf{1}\{\cdot\}$ (indicator function) equals 1 when the event inside braces occurs and 0 otherwise, and $d(v_k, 0)$ is the distance from customer v_k to the depot.

Computing the Expected Recourse Cost II

Worked Example: 3 Customers, $Q = 10$

Suppose customer demands are independent: $\xi_1 \sim \text{Uniform}(2, 6)$, $\xi_2 \sim \text{Uniform}(3, 7)$, $\xi_3 \sim \text{Uniform}(1, 5)$, with means $\mu_1 = 4, \mu_2 = 5, \mu_3 = 3$.

Expected total demand $= 4 + 5 + 3 = 12 > Q = 10$, so *at least one restocking trip is expected*. After visiting customers 1 and 2, the expected cumulative demand is 9, which is close to $Q = 10$. The probability of needing a restock after customer 2 is $P(\xi_1 + \xi_2 > 10) = P(D_2 > 10)$. With ξ_1, ξ_2 uniform on $[2, 6]$ and $[3, 7]$, we have $D_2 \in [5, 13]$; computing the convolution gives $P(D_2 > 10) \approx 0.28$. So with probability 28% there is a recourse trip after customer 2.

Computing the Expected Recourse Cost III

For general demand distributions, the expectation is computed by integration (continuous distributions) or summation (discrete distributions) over all demand scenarios. When customer demands are independent, the computation separates:

Separability for Independent Demands

If $\xi_1, \ldots, \xi_n$ are *independent* random variables, the expected recourse cost can be decomposed route-by-route and even customer-by-customer using convolutions of the individual demand distributions. This separability is crucial for the efficiency of exact and heuristic algorithms.

For the special case of *known demand levels* (a discrete distribution over a finite set of scenarios $S = \{s_1, s_2, \ldots, s_{|S|}\}$ each with probability p_s), the expected recourse cost simplifies to:

$$\mathbb{E}[Q(\sigma, \boldsymbol{\xi})] \;=\; \sum_{s \in S} p_s \, Q(\sigma, \boldsymbol{\xi}^s)$$

where $\boldsymbol{\xi}^s$ is the demand vector in scenario s and $Q(\sigma, \boldsymbol{\xi}^s)$ is the deterministic recourse cost in that scenario.

The Integer L-Shaped Algorithm I

The most important exact algorithm for the a priori SVRP (under stochastic demands) is the **Integer L-Shaped Algorithm**, proposed by Laporte, Louveaux, and van Hamme (1992). It is an extension of the classical *L-shaped method* of stochastic programming to problems with binary (integer) first-stage variables.

Why "L-Shaped"?

The name comes from the shape of the feasible region in a Benders decomposition. The idea is to reformulate the mixed-integer stochastic program as a *master problem* (choosing the tour) plus a *subproblem* (computing the recourse cost for a given tour). Cuts are added to the master problem to approximate the recourse cost function from below.

The Integer L-Shaped Algorithm II

High-Level Structure

1. **Initialize**: Set the master problem (the VRP with no recourse cost approximation) and an upper bound $UB = +\infty$.
2. **Solve the master problem** (the LP relaxation or integer relaxation) to obtain a candidate tour σ.
3. **Evaluate** the exact expected recourse cost $\mathbb{E}[Q(\sigma, \boldsymbol{\xi})]$ for this tour.
4. **Update** the upper bound: if the total cost of σ plus its recourse cost is better than UB, update UB.
5. **Generate cuts**: add optimality cuts (lower bounds on the recourse cost) to the master problem to prevent it from revisiting tours with high recourse cost.
6. **Repeat** until the master lower bound equals UB (optimality gap is zero).

The Integer L-Shaped Algorithm III

Optimality Cuts (L-Shaped Cuts)

For a given candidate solution $\hat{x}$ (the current tour expressed as arc variables), an optimality cut has the form:

$$\theta \geq \alpha(\hat{x}) + \sum_{(i,j)\in E} \beta_{ij}(\hat{x})\,(x_{ij} - \hat{x}_{ij})$$

where:

- θ (theta) is the recourse cost estimate variable added to the master objective.
- $\alpha(\hat{x})$ is the exact recourse cost of tour $\hat{x}$.
- $\beta_{ij}(\hat{x})$ are subgradient (slope) coefficients computed from the recourse function.
- $(x_{ij} - \hat{x}_{ij})$ measures how far the next candidate tour differs from the current one.

Each cut says: "for any tour that differs from $\hat{x}$ in the given way, the recourse cost is *at least* this much."

The Integer L-Shaped Algorithm IV

Full Pseudocode: Integer L-Shaped Algorithm

Input: Graph G, demand distributions $\{\xi_i\}$, capacity Q.
Output: Optimal a priori tour σ^* and its expected cost.

Step 1. Formulate the master problem as a VRP with an extra variable $\theta \geq 0$ approximating the recourse cost. Initialize with no optimality cuts.
Step 2. Solve the master problem. Let $\hat{x}$ be the optimal solution and $\hat{z}$ its objective value ($\hat{z}$ = base route cost + current θ estimate).
Step 3. Compute the exact expected recourse cost $R(\hat{x}) = \mathbb{E}[Q(\hat{x}, \boldsymbol{\xi})]$ by summing over all demand realizations (or using the separability formula).
Step 4. Let z^* = (base cost of $\hat{x}$) $+R(\hat{x})$. If $z^* < UB$, set $UB = z^*$ and $\sigma^* = \hat{x}$.
Step 5. If $\hat{z} \geq UB - \varepsilon$ (the master lower bound is within tolerance of the upper bound): **STOP**, σ^* is optimal.
Step 6. Generate and add an optimality cut for $\hat{x}$ (using the gradient $\nabla R(\hat{x})$ or a subgradient approximation). Optionally add integer L-shaped cuts (to cut off infeasible and suboptimal integer solutions directly).
Step 7. Go to Step 2.

Integer L-Shaped Cuts

The Branch-and-Price Algorithm I

Branch-and-price is a **column generation** method for the set partitioning formulation of the SVRP. It is the most powerful exact approach for larger instances.

Why Column Generation?

The set partitioning model has a binary variable λ_r for every feasible route. The number of feasible routes is exponential in n, so we cannot enumerate them all explicitly. *Column generation* starts with a small subset of routes and adds new routes (columns) only when they can improve the current solution.

The Branch-and-Price Algorithm II

The Pricing Subproblem

At each iteration of column generation, we solve a **pricing subproblem**: "Find the route r with the most negative *reduced cost*":

$$\bar{c}_r \;=\; c_r + \mathbb{E}[Q_r(\xi)] \;-\; \sum_{i \in r} \mu_i$$

where μ_i (mu sub i) are the dual variables (shadow prices) of constraint (8.11) from the LP relaxation. A route with $\bar{c}_r < 0$ can improve the current LP solution and should be added.
The pricing subproblem is itself an NP-hard shortest-path problem with resource constraints (capacity), but effective dynamic-programming algorithms exist for it.

The Branch-and-Price Algorithm III

Branch-and-Price Algorithm Steps

Step 1. Initialize with a few feasible routes (e.g., one per customer: the depot – customer i – depot route).
Step 2. Solve the LP relaxation of the set partitioning model with the current column set.
Step 3. Solve the pricing subproblem to find routes with negative reduced cost. If none exist: the current LP solution is optimal for this node (LP lower bound certified).
Step 4. If the LP solution is integer (all $\lambda_r \in \{0, 1\}$), record the feasible solution (update upper bound). Otherwise, branch on a fractional λ_r: create two sub-problems with $\lambda_r = 0$ (route r excluded) or $\lambda_r = 1$ (route r must be used).
Step 5. Apply bounding: prune any tree node whose LP lower bound exceeds the current upper bound.
Step 6. Return to Step 2 for the next open node.

The computational results reported in the chapter show that branch-and-price can solve instances with up to 50 customers to optimality, which is the state-of-the-art for exact SVRP methods.

Benchmark Results for A Priori Algorithms I

Computational experiments in the chapter use instances derived from the well-known VRP benchmarks of Augerat (1995), with demands replaced by independent uniform random variables.

Benchmark Results: A Priori SVRP Algorithms
(Illustrative values based on chapter discussion; N = number of customers)

Instance	N	#Routes	RAM (opt)	B&P (Gendreau)	ILS	% Gap ILS/opt
Set A-n32	32	3	2000	2021	2034	1.70%
Set A-n33	33	3	1560	1563	1581	1.35%
Set B-n34	34	3	1780	1780	1803	1.29%
Set B-n41	41	4	2560	2571	2598	1.48%
Set B-n78	78	8	8024	8091	8156	1.65%

Figure: Illustrative benchmark results for a priori SVRP algorithms. “RAM” is the exact integer L-shaped method of Laporte, Louveaux, and van Hamme. “B&P” is branch-and-price. “ILS” is iterated local search (metaheuristic). “% Gap” is how far the heuristic is from the exact optimum.

Benchmark Results for A Priori Algorithms II

Takeaway

The exact methods are limited to small instances (up to 30–50 customers) due to the exponential complexity of enumerating demand scenarios and solving the pricing subproblem. Metaheuristics such as ILS and tabu search find near-optimal solutions (within 2–3%) for instances with hundreds of customers. The gap between exact and heuristic solutions is the motivation for ongoing research into tighter bounds and faster subproblem solvers.

Online Correction: The Reoptimization Model I

The reoptimization model (also called the *online* or *dynamic correction* model) takes a different perspective from a priori optimization: instead of committing to a fixed tour in advance, it allows *route corrections during execution* as information is revealed.

The Core Idea

At each stage of the delivery process (each time the vehicle arrives at a customer or the depot), the dispatcher observes the new information (actual demand revealed, customer present or absent) and *solves an optimization problem* to determine the best remaining route. Decisions are made sequentially, adapting to what has been learned.

This is in direct contrast with the a priori approach, where the sequence of customers is fixed and only a pre-specified recourse rule is applied.

Online Correction: The Reoptimization Model II

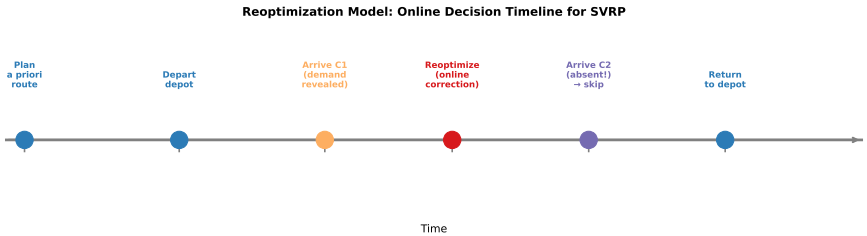


Figure: Timeline of decisions in the reoptimization model. At each event (arrival at a customer, demand revealed, customer absent) a new optimization is performed to update the plan.

Online Correction: The Reoptimization Model III

Markov Decision Process (MDP) Structure

The reoptimization model is naturally formulated as a **Markov Decision Process**:

- **State** s_k: the information available at step k (current location of vehicle, remaining customers to serve, realized demands so far, current vehicle load).
- **Action** a_k: the next customer to visit (or the decision to return to the depot to restock).
- **Transition**: moving from state s_k to s_{k+1} by taking action a_k, with the next customer's demand then revealed.
- **Cost**: the travel cost of the chosen arc, plus any recourse cost incurred.

The goal is to find an *optimal policy* π^* (pi star, pronounced "pie") mapping states to actions so as to minimize the total expected cost.

Online Correction: The Reoptimization Model IV

Why MDP Is Hard to Solve Exactly

The state space of the MDP grows *exponentially* with the number of customers n. For n customers, the state must encode which customers remain to be served (there are 2^n subsets), plus the current vehicle load, plus the current location. This makes exact MDP solution feasible only for very small n (say, $n \leq 10$).

Online Correction: The Reoptimization Model V

Practical Reoptimization Policies

For practical use, several efficient *approximate* reoptimization policies have been studied:

1. **Nearest neighbour**: after each demand revelation, always visit the closest remaining unserved customer. Simple but often suboptimal.
2. **Rollout heuristic**: at each step, use a greedy or simple heuristic to evaluate each possible next customer, choosing the one with the smallest estimated remaining route cost.
3. **Re-solve with deterministic approximation**: after each demand revelation, replace remaining random demands with their expected values and solve the resulting deterministic VRP. This is fast and often produces excellent results.

Online Correction: The Reoptimization Model VI

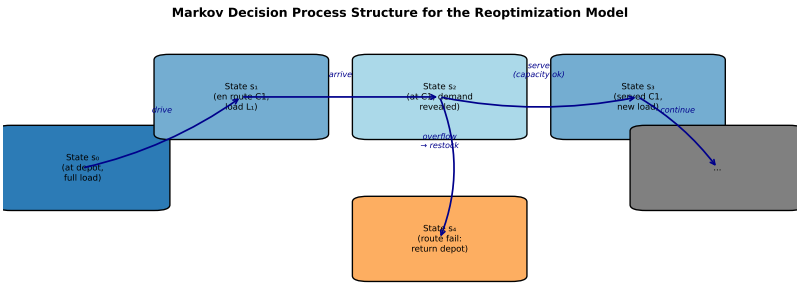


Figure: Markov Decision Process structure for the reoptimization model. Each node is a state (vehicle location, remaining customers, load); edges are decisions with random demand transitions.

Online Correction: The Reoptimization Model VII

Key Comparison with A Priori

The reoptimization model generally achieves **lower expected cost** than a priori optimization because it uses information as it becomes available. However, it requires real-time computation and communication infrastructure, and the plan cannot be communicated to customers in advance. The right choice depends on the operational context.

Probabilistic Formulation: Expected-Cost Objective I

The *probabilistic formulation* of the SVRP is the most theoretically elegant approach. Instead of specifying a recourse policy after the fact, it directly builds the expectation over all random scenarios into the objective function.

Two-Stage Stochastic Programming View

The SVRP under the probabilistic formulation is a **two-stage stochastic program**:

- **First stage** ("here and now" decisions): choose the route plan $\mathbf{x}$ (arc variables) before any demands are revealed. These decisions are made under uncertainty.
- **Second stage** ("wait and see" decisions): after the demand vector $\boldsymbol{\xi}$ is realized, incur the recourse cost $Q(\mathbf{x}, \boldsymbol{\xi})$ to handle any capacity violations.

Probabilistic Formulation: Expected-Cost Objective II

Expected Value Objective

Let Ξ (uppercase Xi, pronounced "ksee") be the support of the random demand vector $\boldsymbol{\xi}$ (the set of all values it can take). The probabilistic formulation minimizes:

$$z^* = \min_{\mathbf{x} \in \mathcal{X}} \sum_{(i,j) \in E} c_{ij}\, x_{ij} + \int_{\Xi} Q(\mathbf{x}, \boldsymbol{\xi})\, f(\boldsymbol{\xi})\, d\boldsymbol{\xi}$$

where:

- $\mathcal{X}$ (calligraphic X) is the set of all feasible first-stage route plans (VRP constraints).
- $f(\boldsymbol{\xi})$ is the joint probability density (or mass function) of all customer demands.
- The integral $\int_{\Xi} \cdots d\boldsymbol{\xi}$ averages the recourse cost over all possible demand outcomes, weighted by their probabilities.

Probabilistic Formulation: Expected-Cost Objective III

How to Read This Formula

The formula says: "choose the route plan that minimizes the sum of (a) the base route cost and (b) the *average* extra cost across all possible demand scenarios." A plan that is cheap deterministically but frequently causes capacity violations will have a high expected recourse cost and will *not* be selected.

Probabilistic Formulation: Expected-Cost Objective IV

Sample Average Approximation (SAA)

For most practical distributions, the integral cannot be computed analytically. The **sample average approximation** (SAA) replaces the integral with an average over a finite sample of S scenarios:

$$\hat{z}_S = \min_{\mathbf{x} \in \mathcal{X}} \sum_{(i,j) \in E} c_{ij} x_{ij} + \frac{1}{S} \sum_{s=1}^{S} Q(\mathbf{x}, \boldsymbol{\xi}^s)$$

where $\boldsymbol{\xi}^1, \ldots, \boldsymbol{\xi}^S$ are independently sampled demand scenarios. As $S \to \infty$ (as S grows to infinity), $\hat{z}_S \to z^*$ (converges to the true optimal value) with probability 1.
In practice, S between 100 and 1000 scenarios is often sufficient for a good approximation.

Probabilistic Formulation: Expected-Cost Objective V

Single-Location Adaptation (Laporte, Louveaux, Mercure 1992)

A special case studied extensively in the literature is the **single-vehicle probabilistic formulation** (VRPSD with one vehicle). Here the model simplifies to:

$$\min_{\sigma} \underbrace{\sum_{k=0}^{n} d(\sigma_k, \sigma_{k+1})}_{\text{base tour cost}} + \underbrace{\mathbb{E}[B(\sigma, \boldsymbol{\xi})]}_{\text{expected recourse}}$$

where $\sigma_0 = \sigma_{n+1} = 0$ (depot) and $B(\sigma, \boldsymbol{\xi})$ is the total return-trip cost for all restocking events.

Stochastic Demands: The VRPSD I

The **Vehicle Routing Problem with Stochastic Demands (VRPSD)** is the most widely studied SVRP variant. Customer demands are random variables; all other problem data (customer locations, number of vehicles, vehicle capacity) are deterministic.

Problem Statement

We are given:

- n customers with *random* demands $\xi_1, \xi_2, \ldots, \xi_n$.
- Demand ξ_i has known mean $\mu_i = \mathbb{E}[\xi_i]$ (mu sub i) and known distribution (often Normal or Poisson).
- Each vehicle has capacity Q.
- Demands are revealed only when the vehicle arrives at a customer.

The goal is to find a set of a priori routes that minimizes the total expected distance (base route cost plus expected recourse cost).

Stochastic Demands: The VRPSD II

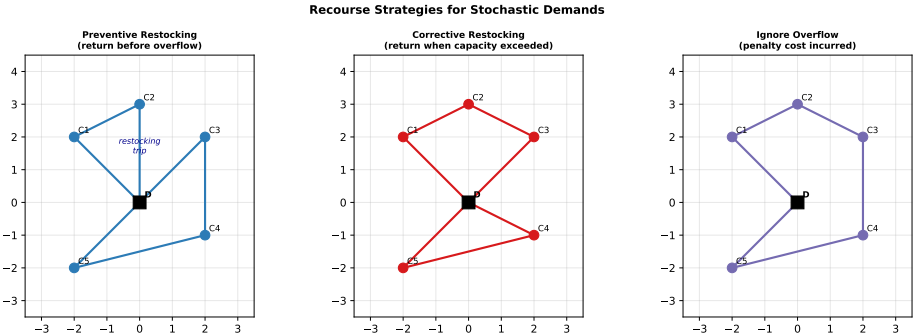


Figure: Three recourse strategies for stochastic demands. Left: preventive restocking (return to depot before capacity is exceeded, as a precaution). Centre: corrective restocking (return only when capacity is actually exceeded). Right: no restocking (incur a penalty cost for overflow).

Stochastic Demands: The VRPSD III

Chance Constraints for VRPSD

An alternative to recourse is to impose a **chance constraint**: require that the probability of a capacity violation on each route is at most ε (epsilon, a small number like 0.05):

$$P\left(\sum_{i \in S_r} \xi_i > Q\right) \leq \varepsilon \quad \forall \text{ route } r$$

Here S_r is the set of customers on route r, and ε is the maximum allowed probability of route failure. This constraint says: "the chance that the total demand on route r exceeds the vehicle capacity must be no greater than ε". The route is *robust* if ε is small (e.g. 5%).

Stochastic Demands: The VRPSD IV

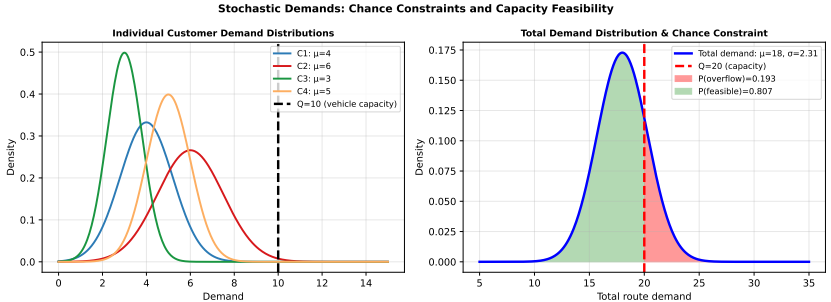


Figure: Left: individual demand distributions for four customers. Right: the total route demand distribution and the chance constraint threshold (vehicle capacity Q). The red area is P(overflow); the green area is P(feasible).

Stochastic Demands: The VRPSD V

Recourse Strategy Details: Preventive vs. Corrective

Let L_k (L sub k) denote the vehicle load after visiting the first k customers in the planned order.

Preventive restocking: the vehicle returns to the depot *before* visiting customer k if the expected remaining demand from customer k onwards would cause overflow. Formally, return if $L_{k-1} + \mu_k > Q$. This is conservative but avoids route failure.

Corrective restocking: the vehicle drives to customer k and, if $L_{k-1} + \xi_k > Q$ (actual demand causes overflow), it returns to the depot to reload and then comes back to serve customer k. This is the most common model in the literature, and is what we described in the network flow formulation above.

Emergency restocking: allow partial delivery, deliver as much as the vehicle has left, and then return for the remainder. This adds additional route segments.

Stochastic Demands: The VRPSD VI

The chapter discusses the following algorithmic approaches for VRPSD:

1. **Exact methods**: Integer L-shaped algorithm (Laporte, Louveaux, van Hamme, 1994) and branch-and-price (Gendreau, Laporte, Seguin, 1996). Can solve instances with up to 50 customers optimally.
2. **Tabu search metaheuristic** (Gendreau, Laporte, Seguin, 1996): a neighbourhood search where moves are "banned" (tabu) for a short time after being applied, to escape local optima. Produces solutions within 1–2% of optimum for instances with up to 100 customers.
3. **Simulated annealing**: accepts some worsening moves with a probability that decreases over time (the analogy is to slow cooling in metallurgy), allowing escape from local optima.
4. **Genetic algorithms**: maintain a *population* of candidate solutions and evolve them through crossover (combining parts of two solutions) and mutation (small random changes).

Stochastic Demands: The VRPSD VII

Key Insight

Even for moderate-sized instances ($n \approx 50$), the exact expected recourse cost requires integrating over all demand scenarios, which is computationally expensive. The main contribution of the integer L-shaped algorithm is to avoid evaluating the full expected recourse cost at every iteration, instead using *lower bounding cuts* that are much cheaper to compute.

Python: Simulating Expected Recourse Cost (VRPSD)

```python
import numpy as np

def simulate_recourse_cost(route, demands_samples, depot_dists, Q):
    """
    Estimate E[Q(sigma, xi)] by Monte Carlo simulation.

    Parameters
    ----------
    route          : list of customer indices (0-indexed), e.g. [0,2,4,1,3]
    demands_samples : 2D array of shape (S, n), where S = number of scenarios
                      and n = number of customers.  demands_samples[s][i] is the
                      demand of customer i in scenario s.
    depot_dists    : 1D array of length n; depot_dists[i] = distance depot<->customer i
    Q              : vehicle capacity (maximum load)

    Returns
    -------
    expected_recourse : float, the estimated E[Q(sigma, xi)]
    """
    S = demands_samples.shape[0]
    total_recourse = 0.0

    for s in range(S):
        xi = demands_samples[s]    # demand vector for this scenario
        load = 0.0
        recourse_cost = 0.0
        for cust in route:
```

Stochastic Customers: The VRPSC I

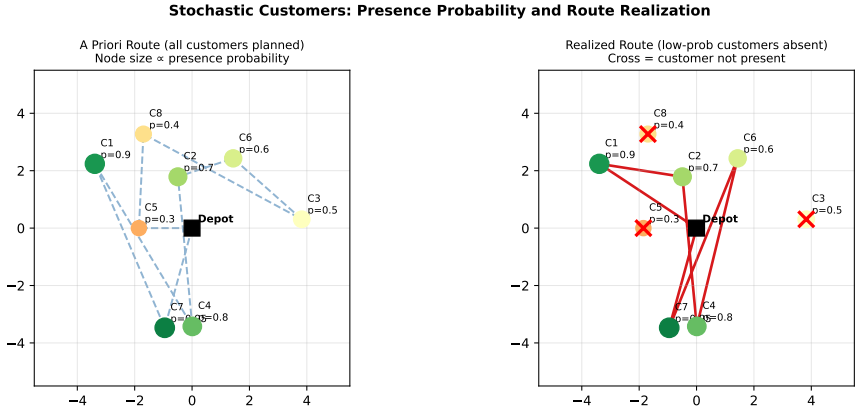
The **Vehicle Routing Problem with Stochastic Customers (VRPSC)** considers the case where it is uncertain which customers will actually need service. Each customer i is present (has a demand) independently with probability p_i (p sub i), and absent with probability $1 - p_i$.

Difference from VRPSD

In the VRPSD, every customer is definitely present but has an unknown demand. In the VRPSC, the demand per customer is fixed (say, one unit), but the set of customers to serve is random. Customers who are absent when the vehicle arrives are simply skipped, and the vehicle continues to the next customer in its planned tour.

This problem was first studied in the context of the **Probabilistic Travelling Salesman Problem (PTSP)**, introduced by Jaillet (1985) and Bertsimas (1988). The PTSP is the single-vehicle version.

Stochastic Customers: The VRPSC II



Stochastic Customers: The VRPSC III

Figure: Left: the a priori tour including all customers, with node size proportional to presence probability. Right: one realization where low-probability customers are absent (marked with red cross); the vehicle skips them following the a priori order.

The Single-Location Model (VRPSC with One Vehicle)

Consider a single vehicle based at a depot 0. Customers are located at positions $v_1, \ldots, v_n$, each present independently with probability p_i. Let $\sigma = (\sigma_1, \sigma_2, \ldots, \sigma_n)$ be the a priori tour.
Given that customer σ_k is present, the vehicle visits it; otherwise it drives directly to the next present customer in the sequence. The expected cost of the a priori tour is:

$$\mathbb{E}[C(\sigma)] = \sum_{i=1}^{n} \sum_{j>i} p_{\sigma_i} p_{\sigma_j} \prod_{k=i+1}^{j-1} (1 - p_{\sigma_k}) \cdot d(\sigma_i, \sigma_j)$$

This sum accounts for all pairs (σ_i, σ_j) of present customers where σ_j is the successor of σ_i in the realized tour (i.e., all customers between them in the a priori order are absent). The product $\prod_{k=i+1}^{j-1}(1 - p_{\sigma_k})$ is the probability that all intermediate customers are absent.

Stochastic Customers: The VRPSC V

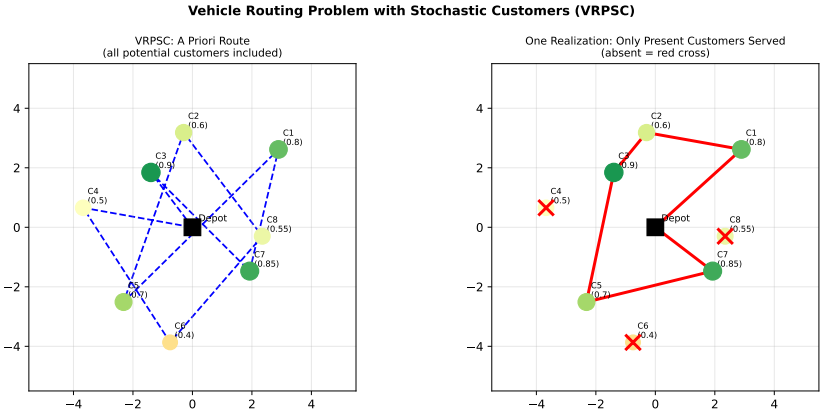


Figure: VRPSC single-vehicle illustration. Left: the a priori tour (planned for all customers). Right: one realization (only high-probability customers are present; low-probability customers are skipped).

Worked Example: Expected VRPSC Cost with 3 Customers

Suppose the depot is at position 0, and there are 3 customers with positions A, B, C at unit distances: $d(0, A) = 1, d(A, B) = 1, d(B, C) = 1, d(C, 0) = 1, d(0, B) = \sqrt{2}, \ldots$
Say the a priori tour is $0 \to A \to B \to C \to 0$ and $p_A = 0.9, p_B = 0.6, p_C = 0.8$.
With probability $p_A \cdot p_B \cdot p_C = 0.9 \times 0.6 \times 0.8 = 0.432$, all three are present and the tour is $0 \to A \to B \to C \to 0$.
With probability $p_A \cdot (1 - p_B) \cdot p_C = 0.9 \times 0.4 \times 0.8 = 0.288$, B is absent: tour becomes $0 \to A \to C \to 0$ (saving the $A \to B$ and $B \to C$ edges, but paying $d(A, C)$ instead).
Summing over all $2^3 = 8$ realizations gives the expected tour cost. Optimizing over all $n!/2$ possible a priori tours yields the optimal VRPSC solution.

Stochastic Customers: The VRPSC VII

Solution Approaches for VRPSC / PTSP

1. **Exact branch-and-bound**: for small n (up to 15). The expected cost of each tour is computed explicitly by summing over all 2^n realizations.
2. **Savings-based heuristic**: apply the Clarke-Wright savings algorithm but using *expected savings* $\mathbb{E}[s_{ij}]$ rather than deterministic savings. An expected saving $\mathbb{E}[s_{ij}]$ accounts for the probability that both customers i and j are present.
3. **Tabu search on a priori tours**: neighbourhood moves (2-opt, 3-opt, or-opt) are applied to permutations, with the expected cost evaluated at each step.
4. **Genetic algorithm**: maintain a population of permutations; crossover and mutation operators are designed to preserve good sub-sequences.

The chapter reports that tabu search algorithms for the VRPSC can find solutions within 1% of optimal for instances with up to 100 customers.

Stochastic Customers: The VRPSC VIII

Key Insight: Order Matters

Even though absent customers are simply skipped, the *order* in which customers appear in the a priori tour affects the expected cost. Placing two customers with low presence probabilities next to each other is risky: if both are absent, the vehicle makes a long jump. The optimal a priori tour tends to interleave high- and low-probability customers.

Python: Expected Tour Cost for VRPSC

```python
import numpy as np
from itertools import permutations

def expected_ptsp_cost(tour, probs, dist_matrix):
    """
    Compute the exact expected tour cost for the PTSP (single-vehicle VRPSC).

    Parameters
    ----------
    tour        : list, e.g. [0, 1, 2, 3] (0 = depot; others are customers)
    probs       : dict {customer_id: presence_probability}
                  e.g. {1: 0.9, 2: 0.6, 3: 0.8}
    dist_matrix : 2D array; dist_matrix[i][j] = distance from i to j

    Returns
    -------
    E_cost : float, expected total tour distance
    """
    customers = [t for t in tour if t != 0]    # exclude depot
    n = len(customers)
    E_cost = 0.0

    # Enumerate all 2^n subsets of customers that could be present
    for mask in range(1 << n):
        prob_scenario = 1.0
        present = [0]  # depot always included
        for k, c in enumerate(customers):
```

Stochastic Travel Times: Time-Window Feasibility I

The **VRP with Stochastic Travel Times (VRPST)** (also called the VRP with stochastic service times) addresses the situation where the time to travel between two locations is uncertain, typically due to variable traffic conditions.

What Makes This Different?

In the VRPSD and VRPSC, stochasticity affects *load feasibility* (whether the vehicle can carry all demanded goods). In the VRPST, stochasticity affects *time feasibility*: a customer may have a time window $[a_i, b_i]$ (earliest and latest arrival time), and the vehicle may arrive too early (must wait, incurring a waiting cost) or too late (violates the time window, incurring a penalty).

Stochastic Travel Times: Time-Window Feasibility II

Two Components of Stochastic Service Time

The total time to serve customer i after departing from customer j has two random components:

1. **Travel time** t_{ij} (t sub i-j): random, affected by traffic, weather, road conditions.
2. **Service time** s_i (s sub i): random, affected by how long the unloading takes, customer's readiness, etc.

The sum $T_{ij} = t_{ij} + s_i$ is the total stochastic time from departing j to completing service at i.

Stochastic Travel Times: Time-Window Feasibility III

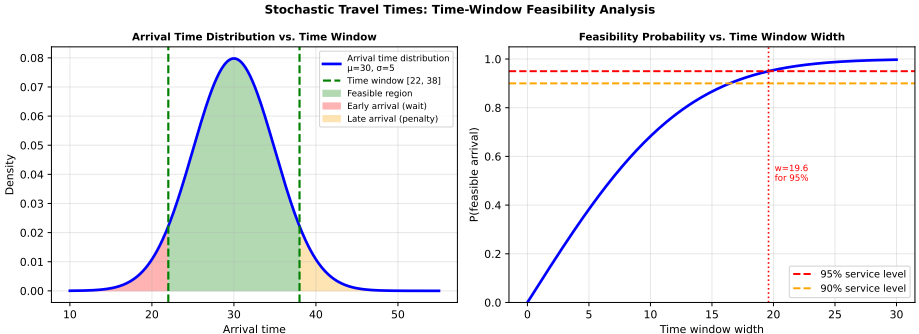


Figure: Left: the arrival time distribution (bell curve) at a customer relative to the time window $[a, b]$. Green area = on-time probability. Red area = early arrival (vehicle must wait). Orange area = late arrival (time-window violation, penalty). Right: the probability of feasible arrival increases as the time window width grows.

Stochastic Travel Times: Time-Window Feasibility IV

Objective Function with Expected Penalties

The VRPST objective minimizes expected total route cost including waiting and late-arrival penalties:

$$\min_{\mathbf{x}} \sum_{(i,j)} c_{ij} x_{ij} + \alpha \sum_{i} \mathbb{E}[\max(0,\, a_i - A_i)] + \beta \sum_{i} \mathbb{E}[\max(0,\, A_i - b_i)]$$

where:

- A_i (A sub i) — the *random arrival time* at customer i (it is random because travel times are random).
- a_i (a sub i) — the *earliest* acceptable arrival time at customer i (left bound of time window).
- b_i (b sub i) — the *latest* acceptable arrival time at customer i (right bound of time window).
- α (alpha) — the penalty per unit time of early arrival (waiting cost per unit time).
- β (beta) — the penalty per unit time of late arrival (larger than α in practice: missing a time window is more costly than waiting).
- $\max(0, a_i - A_i)$ — the positive waiting time (zero if arrival is not early).
- $\max(0, A_i - b_i)$ — the positive lateness (zero if arrival is within the time window).

How to Read the VRPST Objective

Conclusions I

This chapter has surveyed the three main classes of Stochastic Vehicle Routing Problems and the principal solution methods for each.

Conclusions II

What We Have Covered

1. **A priori optimization**: build a complete route plan before uncertainty is resolved, evaluate it by its expected cost under a fixed recourse policy (re-supply when capacity is exceeded). Key algorithms: integer L-shaped method, branch-and-price.
2. **Reoptimization model**: adapt the route plan online as information is revealed. Formulated as a Markov Decision Process; solved approximately by rollout heuristics and re-solve policies.
3. **Probabilistic (expected-cost) formulation**: minimize the true expected total cost over all random scenarios; approximated by Sample Average Approximation.
4. **Stochastic demands (VRPSD)**: uncertainty in customer demands; handle by chance constraints or recourse (preventive or corrective restocking).
5. **Stochastic customers (VRPSC/PTSP)**: uncertainty in which customers are present; optimize a priori tour order to minimize expected distance when absent customers are skipped.
6. **Stochastic travel times (VRPST)**: uncertainty in travel and service times; time-window feasibility becomes probabilistic; objective includes expected waiting and late-arrival penalties.

Conclusions III

Summary Comparison: Three Main SVRP Variants

	Stochastic Demands	Stochastic Customers	Stochastic Travel Times
Uncertain parameter	Customer demand d_i	Customer presence y_i	Arc travel time t_{ij}
Typical recourse	Restocking or penalty	Skip absent customers	Wait / accept late penalty
Solution approach	Chance constraints	VRPSD + stoch prog	Robust scheduling
Key challenge	Capacity violation	Expected cost	Time window feasibility

Conclusions IV

Figure: Summary comparison of the three main SVRP variants: uncertain parameter, typical recourse action, solution approach, and key challenge.

Conclusions V

Future Research Directions

The chapter identifies the following open problems and research directions as of the time of writing (2014):

1. **Multi-period and multi-echelon SVRP**: combining stochasticity with multiple depots, multiple time periods, or multi-level supply chains.
2. **Integration with inventory management**: joint optimization of routing and replenishment decisions when customer demand is stochastic over time.
3. **Richer uncertainty models**: correlated demands (e.g., seasonal effects causing nearby customers' demands to be positively correlated), non-stationary distributions, and demand learning over time.
4. **Robust and distributionally robust formulations**: optimizing for the worst-case distribution within a class (e.g., all distributions with given mean and variance), protecting against model misspecification.
5. **Large-scale heuristics**: most current exact methods are limited to $n \leq 50$. Developing effective metaheuristics and approximation algorithms for $n \geq 200$ is an important practical goal.
6. **Dynamic VRP**: combining stochastic modelling with real-time information updates from GPS, traffic monitoring, and demand signals.
7. **Machine learning integration**: using historical data to learn demand distributions, predict customer presence, and warm-start optimization algorithms.

Selected Bibliography I

The following works are cited in this chapter. References are grouped by topic for easier navigation.

A Priori Optimization and Integer L-Shaped Method:

- Laporte, G., Louveaux, F.V., and van Hamme, L. (1994). An integer L-shaped algorithm for the capacitated vehicle routing problem with stochastic demands. *Operations Research*, 10(1), 10–23.
- Gendreau, M., Laporte, G., and Seguin, R. (1996). Stochastic vehicle routing. *European Journal of Operational Research*, 88(1), 3–12.
- Christiansen, C.H. and Lysgaard, J. (2007). A branch-and-price algorithm for the capacitated vehicle routing problem with stochastic demands. *Operations Research Letters*, 35(6), 773–781.

Probabilistic TSP and Stochastic Customers:

- Jaillet, P. (1988). A priori solution of a travelling salesman problem in which a random subset of the customers are visited. *Operations Research*, 36(6), 929–936.
- Bertsimas, D.J., Jaillet, P., and Odoni, A. (1990). A priori optimization. *Operations Research*, 38(6), 1019–1033.
- Laporte, G., Louveaux, F.V., and Mercure, H. (1989). Models and exact solutions for a class of stochastic location-routing problems. *European Journal of Operational Research*, 39(1), 71–78.

Selected Bibliography II

Stochastic Travel Times:

- Laporte, G., Louveaux, F.V., and Mercure, H. (1992). The vehicle routing problem with stochastic travel times. *Transportation Science*, 26(3), 161–170.
- Kenyon, A.S. and Morton, D.P. (2003). Stochastic vehicle routing with random travel times. *Transportation Science*, 37(1), 69–82.

Metaheuristics for SVRP:

- Gendreau, M., Laporte, G., and Seguin, R. (1996). A tabu search heuristic for the vehicle routing problem with stochastic demands and customers. *Operations Research*, 44(3), 469–477.
- Secomandi, N. (2000). Comparing neuro-dynamic programming algorithms for the vehicle routing problem with stochastic demands. *Computers & Operations Research*, 27(11–12), 1201–1225.
- Hvattum, L.M., Lokketangen, A., and Laporte, G. (2006). Solving a dynamic and stochastic vehicle routing problem with a sample scenario hedging heuristic. *Transportation Science*, 40(4), 421–438.

Books and Surveys:

- Birge, J.R. and Louveaux, F. (2011). *Introduction to Stochastic Programming*, 2nd ed. Springer.
- Powell, W.B., Jaillet, P., and Odoni, A. (1995). Stochastic and dynamic networks and routing. In: Ball et al. (eds.), *Handbooks in Operations Research and Management Science: Network Routing*.